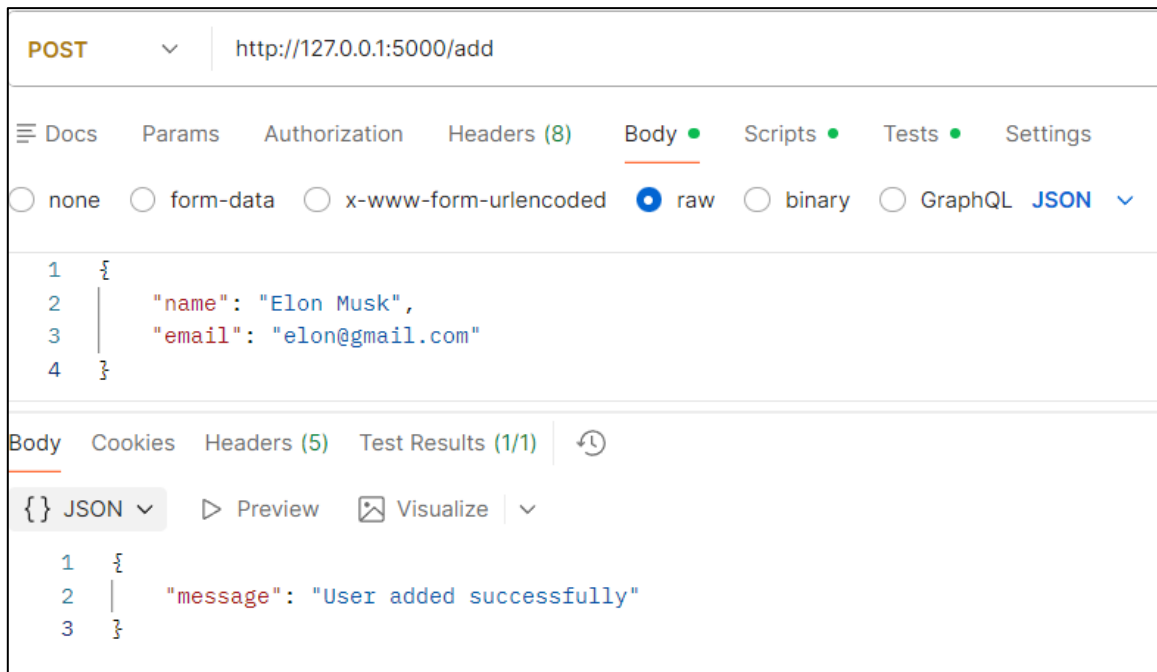


Practical No. 7

Aim: To design and implement a backend REST API using Python Flask and connect it to a SQLite database to perform CRUD (Create, Read, Update, Delete) operations.

Output:



POST http://127.0.0.1:5000/add

Docs Params Authorization Headers (8) Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "name": "Elon Musk",
3   "email": "elon@gmail.com"
4 }
```

Body Cookies Headers (5) Test Results (1/1)

{ } JSON Preview Visualize

```
1 {
2   "message": "User added successfully"
3 }
```



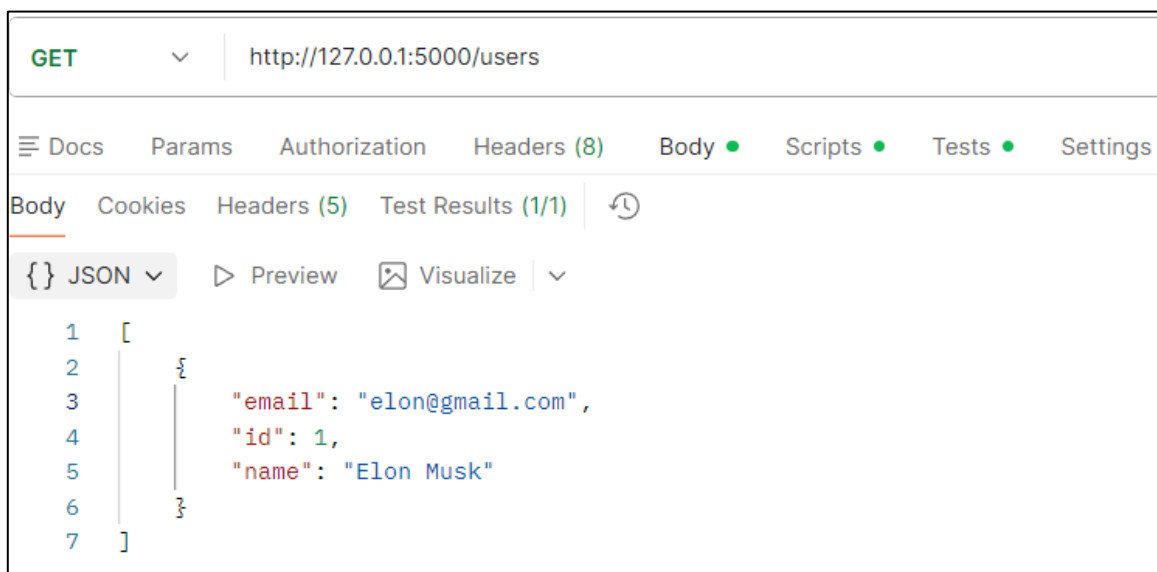
fsd_prac7 > database.db

Filter 2 table: Rows: 1

TABLES

- sqlite_sequence
- users

	id	name	email
1	2	Elon Musk	elon@gmail.com



GET http://127.0.0.1:5000/users

Docs Params Authorization Headers (8) Body Scripts Tests Settings

Body Cookies Headers (5) Test Results (1/1)

{ } JSON Preview Visualize

```
1 [
2   {
3     "email": "elon@gmail.com",
4     "id": 1,
5     "name": "Elon Musk"
6   }
7 ]
```

PUT

http://127.0.0.1:5000/update/1

Docs

Params

Authorization

Headers (8)

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1 {

2 | "name": "Elon Musk SpaceX",

3 | "email": "elon123@gmail.com"

4 }

Body

Cookies

Headers (5)

Test Results (1/1)

{}

JSON

Preview

Visualize

1 {

2 | "message": "User updated successfully"

3 }

fsd_prac7 > database.db

Filter 2 table: Rows: 1

TABLES

sqlite_sequence

users

id	name	email
1	2 Elon Musk SpaceX	elon123@gmail...

DELETE

http://127.0.0.1:5000/delete/1

Docs

Params

Authorization

Headers (8)

Body

Scripts

Tests

Settings

Body

Cookies

Headers (5)

Test Results (1/1)

{}

JSON

Preview

Visualize

1 {

2 | "message": "User deleted successfully"

3 }

Code:**app.py:**

```
from flask import Flask, request, jsonify
import sqlite3
app = Flask(__name__)
# Database connection function
def get_db_connection():
    conn = sqlite3.connect("database.db")
    conn.row_factory = sqlite3.Row
    return conn

# Create table
conn = get_db_connection()
conn.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL
)
""")
conn.commit()
conn.close()
# Home route
@app.route("/")
def home():
    return "Flask Backend API is Running"
# CREATE user
@app.route("/add", methods=["POST"])
def add_user():
    data = request.json
    conn = get_db_connection()
    conn.execute(
        "INSERT INTO users (name, email) VALUES
        (?, ?)",
        (data["name"], data["email"])
    )
    conn.commit()
    conn.close()
    return jsonify({"message": "User added
successfully"})
```

```
# READ users
@app.route("/users", methods=["GET"])
def get_users():
    conn = get_db_connection()
    users = conn.execute("SELECT * FROM
users").fetchall()
    conn.close()
    return jsonify([dict(user) for user in users])
# UPDATE user
@app.route("/update/<int:id>",
methods=["PUT"])
def update_user(id):
    data = request.json
    conn = get_db_connection()
    conn.execute(
        "UPDATE users SET name=?, email=?
WHERE id=?",
        (data["name"], data["email"], id)
    )
    conn.commit()
    conn.close()
    return jsonify({"message": "User updated
successfully"})
# DELETE user
@app.route("/delete/<int:id>",
methods=["DELETE"])
def delete_user(id):
    conn = get_db_connection()
    conn.execute("DELETE FROM users WHERE
id=?", (id,))
    conn.commit()
    conn.close()
    return jsonify({"message": "User deleted
successfully"})
if __name__ == "__main__":
    app.run(debug=True)
```